

쿠창배모-리소스 관리

🕒 작성일시	2026년 8월 16일 오후 6:10
📁 과목	쿠버네티스
☰ 스프린트	

1. 쿠버네티스 스케줄러

쿠버네티스에서는 새로운 Pod가 생성되었다고 아무 Node에나 배치되는 것은 아니다. kube-scheduler는 현재 클러스터에 존재하는 Node들을 확인하여 후보를 확인하고 가장 낫기 좋은 Node에 Pod를 할당한다.

Scheduler의 흐름

1. Pod 생성
2. Pending
3. Scheduler
4. Filter에서 가능한 Node 확인
5. Score에서 가장 최적의 Node 결정
6. 해당 Node에 Pod를 Bind

1] Filter

책에서는 Predicate라고 불렀으며 현재는 Scheduling Framework에서 Filter가 담당한다.

결과는 사실상 true / false로 나온다.

- `true` → 할당 가능: Node 후보
- `false` → 할당 불가능: Node 후보 제외

하나라도 실패하면 Node 후보에서 제거한다.

주요 Filter Plugin

Scheduler Configuration

Feature state: Stable since Kubernetes v1.25 You can customize the behavior of the kube-scheduler by writing a configuration file and passing its path as a command line

 https://kubernetes.io/docs/reference/scheduling/config/?utm_source=chatgpt.com

kuber

과거 Predicate 계열	현재 주로 담당하는 Plugin	확인 내용
CheckNodeUnschedulablePred	NodeUnschedulable	cordon 등으로 스케줄링 금지된 Node인가
PodFitsResourcesPred	NodeResourcesFit	CPU/Memory 등의 request를 받을 수 있는가
PodFitsHostPortsPred	NodePorts	필요한 HostPort를 사용할 수 있는가
PodToleratesNodeTaintsPred	TaintToleration	Node의 Taint를 Pod가 견딜 수 있는가
MatchInterPodAffinityPred	InterPodAffinity	Pod affinity / anti-affinity를 만족하는가
CheckVolumeBindingPred	VolumeBinding	필요한 Volume을 해당 Node에서 사용할 수 있는가

2] Score

책에서는 Priority라고 불렀으며 현재는 Scheduling Framework에서 Score 단계가 담당한다.

Filter를 모두 거쳤을 때 후보 Node가 하나만 남았다면 고민할 것이 없겠지만, 실제로는 여러 Node가 Filter의 조건을 만족할 수 있다.

그렇다면 Scheduler는 "여러 후보들 중에서 어디에 넣는 것이 제일 좋은지"를 판단해야 한다.

Score는 하나의 기준을 보는 것이 아니라 여러 Scoring 기준을 통해 점수를 매겨 총점이 제일 높은 Node에 Pod를 배치하게 된다.

만약 점수가 같다면? 책에서는 round-robin 방식으로 한다고 하지만 공식문서에서는 현재 random으로 나온다.

*If there is more than one node with equal scores, kube-scheduler selects one of these at **random**.*

추후에 나올 NodeAffinity의 weight도 이 Score에 연결된다.

2. 고급 스케줄링 기법

쿠버네티스는 알아서 적절히 스케줄링을 하지만, 가만 **고민해보면 “운영”에서 고려해야 할 부분들은 개발자가 직접 지정을 해줘야 하는 부분들이 어느정도 존재한다.** 크게 노드를 대상으로, 혹은 파드를 중심으로 스케줄링을 설정할 수 있다.

예를 들어 워커 노드가 2개가 존재하고 각 워커 노드의 물리적 위치가 다르게 구성한다면 즉, **가용 영역을 다르게 구성했다면 일반적으로 우리가 원하는 것은 애플리케이션이 하나의 노드에만 쏠리는 것이 아니라 균등하게 있기를 원할 것이다.**

노드 기준, 파드 기준, 테인트, 톨러레이션을 차근차근 알아보자

1] 노드의 레이블을 기준으로 스케줄링

실습환경 노드 k8s-cp-1, k8s-w-1

k8s-cp-1 레이블: `disktype=hdd`, `kubernetes.io/zone=zone-b`

k8s-w-1 레이블: `disktype=ssd`, `kubernetes.io/zone=zone-a`

노드셀렉터: nodeSelector → “해당하는 라벨을 가진 노드로 가고 싶다”

AND 조건만 가능으로 여러 개 쓰면 전부 만족하는 노드만 대상이 된다.

▼ `disktype=ssd` 노드에 파드 설정

```
apiVersion: v1
kind: Pod
metadata:
  name: redis
spec:
  containers:
    - name: redis
      image: redis:alpine
  nodeSelector:
    disktype: ssd    # disktype=ssd 레이블이 있는 노드에
```

```
만
# 따라서 k8s-w-1 노드에 할당
```

▼ disktype=ssd & gpu=true 노드에 파드 설정

```
apiVersion: v1
kind: Pod
metadata:
  name: redis
spec:
  containers:
  - name: redis
    image: redis:alpine
  nodeSelector:
    disktype: ssd
    gpu: "true" # disktype=ssd & gpu="true" 노드에만
# gpu="true" 노드가 없으므로 pending
```

```
# kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
redis-gpu     0/1     Pending   0           26m

# kubectl describe pod redis-gpu | tail -10
Events:
  Type          Reason          Age          From
  Message
  ----
  Warning       FailedScheduling  26m          default-scheduler  0/2 nodes are available: 2 node(s) didn't match Pod's node affinity/selector.
preemption: 0/2 nodes are available: 2 Preemption is not helpful for scheduling.
```

노드 어피니티: nodeAffinity → “노드셀렉터의 확장판”

▼ 예시

```
apiVersion: v1
kind: Pod
metadata:
  name: db-pod
  labels:
    app: database

spec:
  containers:
    - name: mariadb
      image: mariadb:11.4
      env: ...
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: distype
                operator: In
                values:
                  - ssd
                  - nvme
```

• operator

operator	의미
In	값 목록 안에 있으면
NotIn	값 목록에 없으면(제외 조건)
Exists	그 키만 있다면(값 무관)
DoesNotExist	그 키가 없으면
Gt / Lt	값이 크면 / 작으면 (숫자 비교)

▼ Gt / Lt 예시

```
kubectl label node k8s-cp-1 cpu-count=8
kubectl label node k8s-w-1 cpu-count=4
kubectl label node k8s-w-2 cpu-count=16 # 하나 더 있다고 가정
```

```
affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      nodeSelectorTerms:
        - matchExpressions:
            - key: cpu-count
              operator: Gt
              values:
                - "4"
            - key: cpu-count
              operator: Lt
              values:
                - "16"

# k8s-cp-1에만 할당
```

• 조건 행동

- `requiredDuringScheduling` = 배치할 때 필수.
못 지키면 Pending
- `preferredDuringScheduling` = 배치할 때 선호.
못 지켜도 배치는 됨
- `IgnoredDuringExecution` = 실행 중에는 재검사 안함.
이미 도는 파는 규칙이 깨져도 쫓아내지 않음

본래 IgnoredDuringExecution의 쌍으로 RequiredDuringExecution 또한 만들 예정이었지만 현재까지 만들어지고 있지 않고 있다.

▼ 해석해보기

```
spec:
  containers:
    ...
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: disktype
                operator: In
                values:
                  - ssd
                  - nvme
            preferredDuringSchedulingIgnoredDuringExecution:
          - weight: 100
            preference:
              matchExpressions:
                - key: topology.kubernetes.io/zone
                  operator: In
                  values:
                    - zone-a
```

requiredDuringScheduling

```
nodeSelectorTerms:      # OR 관계
- matchExpressions:     # AND 관계
  - key: disktype
    ...
  - key: gpu
    ...
- matchExpressions:
  ...
```

```
requiredDuringSchedulingIgnoredDuringExecution:
```

```
  nodeSelectorTerms:
```

```
    # 조건 그룹 1
```

- matchExpressions:
 - key: disktype:
 - operator: In
 - values:
 - ssd
 - key: gpu
 - operator: In
 - values:
 - "true"

```
    # 조건 그룹 2
```

- matchExpressions:
 - key: node-type
 - operator: In
 - values:
 - high-performance

```
(disktype=ssd AND gpu=true)
```

```
OR
```

```
(node-type=high-performance
```

preferredDuringScheduling

```
preferredDuringSchedulingIgnoredDuringExecution:
```

- weight: 100
 - preference:
 - matchExpressions:
 - key: disktype
 - operator: In
 - values:
 - ssd
- weight: 50
 - preference:

```

    matchExpressions:
      - key: topology.kubernetes.io/zone
        operator: In
        values:
          - zone-a
  - weight: 20
    preference
      matchExpressions:
        - key: node-type
          operator: In
          values:
            - high-performance

```

해당하는 노드마다 점수를 계산하여 가장 높은 노드에 할당

```

k8s-w-1
ssd          +100
zone-a       +50
node-type    +0 (node-type 레이블 할당 X)
-----
150점

k8s-cp-1
hdd          +0
node-type    +0 (node-type 레이블 할당 X)
-----
0점

```

2] 파드를 대상으로 스케줄링

파드 어피니티: podAffinity → “특정 파드 옆에 붙여줘”

▼ redis.yml (app: redis)

```

apiVersion: v1
kind: Pod
metadata:

```

```
name: redis
labels:
  app: redis

spec:
  containers:
  - name: redis
    image: redis:alpine
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-server

spec:
  replicas: 2

  selector:
    matchLabels:
      app: web-server

  template:
    metadata:
      labels:
        app: web-server

    spec:
      containers:
      - name: nginx
        image: nginx:alpine

      affinity:
        podAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
          - labelSelector:
              matchExpressions:
              - key: app
```

```

operator: In
values:
- redis

topologyKey: kubernetes.io/hostname

```

파드 안티-어피니티: podAntiAffinity → “특정 파드 옆에 붙이지 말아줘”

파드 안티-어피니티는 HA 목적으로 종종 쓰인다. 아무런 설정 없이 Deployment의 replicas를 2개 설정하면 하나의 노드에 몰릴 수 있다. 당연히 해당 노드가 죽으면 서비스가 통째로 죽는 상황이 발생한다.

따라서 책에 나온 예시처럼 HA를 위해선 Nginx와 같은 웹서버를 특정 노드에만 넣으면 안 된다.

오히려 모든 노드에 균등하게 들어가야 하나의 노드가 죽더라도 서비스를 유지할 수 있다.

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-server
spec:
  replicas: 2
  selector:
    matchLabels:
      app: web-server
  template:
    metadata:
      labels:
        app: web-server
    spec:
      containers:
        - name: nginx
          image: nginx:alpine
          affinity:
            podAntiAffinity:
              requiredDuringSchedulingIgnoredDuringExecution:
                - labelSelector:
                    matchExpressions:

```

```
- key: app
  operator: In
  values: ["web-server"]
topologyKey: "kubernetes.io/hostname"
```

topologyKey → Node의 구역 설정

podAffinity나 podAntiAffinity 만 생각하게 된다면 “특정 Pod 옆에 붙어라” 혹은 “특정 Pod 옆에 떨어져라”인데 정확하게 그 범위가 정해지지 않았다.

→ Pod를 대상으로 붙어나 떨어지는데 도대체 어느 범위까지 붙고 떨어질지 범위 설정이 필요한데 그것이 바로 topologyKey이다.

만약 쿠버네티스 Node들의 각 라벨이 아래와 같다고 가정해보자.

노드	kubernetes.io/hostname	topology.kubernetes.io/zone
node-1	node-1	zone-a
node-2	node-2	zone-a
node-3	node-3	zone-a
node-4	node-4	zone-b
node-5	node-5	zone-b
node-6	node-6	zone-b

그리고 topologyKey에 따라 구역을 다음과 같이 나눌 수 있다.

- kubernetes.io/hostname → 구역이 6개(값이 전부 다르기 때문에)
- topology.kubernetes.io/zone → 구역 2개(값이 2개로만 나뉘지기 때문에)

주의할 점

- topologyKey는 Node label을 작성한다.
- 해당 label이 없는 Node는 계산에서 제외한다.
- 검사 대상은 “그 구역의 모든 파드”들이다.

3] 테인트(Taint) & 톨러레이션(Toleration)

테인트와 톨러레이션의 의미가 잘 와닿지 않는데, **테인트는 “노드에 붙이는 출입금지 팻말”, 톨러레이션은 “파드가 가진 출입증”**이라고 생각하면 이해가 쉽다.

학창시절에 방송반 학생들만 출입증을 가지고 방송실에 출입할 수 있었고, 방송실 반에는 “관계자외 출입금지” 등의 팻말이 붙어있었다.

테인트: Taint → Node에 팻말 걸기

책의 내용으로 형식을 살펴보자면 아래와 같은 형식으로 구성된다

```
gpu = true : NoSchedule
  ↑       ↑       ↑
key  value  effect
```

실제로 명령어를 통해 노드에 테인트를 걸 수 있다

```
# key/value/effect
kubectl taint nodes k8s-cp-1 gpu=true:NoSchedule

# key/effect
kubectl taint nodes k8s-w-1 maintenance:NoSchedule

# 테인트 확인
kubectl describe node k8s-cp-1 | grep -A5 Taints
```

```
kubectl taint nodes k8s-cp-1 gpu=true:NoSchedule-
```

Label을 지울 때와 마찬가지로 - 를 뒤에 붙여서 테인트를 삭제할 수 있다.

테인트에 따른 효과 설정

- **NoSchedule**
 - 출입증이 없는 새 파드는 못들어온다. 단, 이미 안에 있던 파드는 그대로 둔다.
- **PreferNoSchedule**
 - 가급적으로 다른 곳으 가주고, 어쩔 수 없는 상황에서만 노드 선택
- **NoExecute**

- 출입증이 없는 새 파드도 못들어오고, 이미 돌고 있던 파드 중에도 출입증이 없으면 쫓아낸다.
- 쫓겨난 파드는 Deployment나 ReplicaSet에 소속된 파드라면 다른 노드에 새로 만들어준다.

톨러레이션:Toleration → Pod에 출입증 만들기

톨러레이션은 파드 YAML spec 아래에 작성합니다

▼ Pod

```
apiVersion: v1
kind: Pod
metadata:
  name: ai-training
spec:
  containers:
    - name: trainer
      image: tensorflow:latest
  tolerations:
    - key: "gpu"
      operator: "Equal"
      value: "true"
      effect: "NoSchedule"
```

▼ Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ai-training
spec:
  replicas: 2

  selector:
    matchLabels:
      app: ai-training
```

```

template:
  metadata:
    labels:
      app: ai-training

  spec:
    containers:
      - name: trainer
        image: tensorflow:latest

    tolerations:
      - key: "gpu"
        operator: "Equal"
        value: "true"
        effect: "NoSchedule"

```

- **key** — 어떤 팻말에 대한 출입증인지 명시(Node의 Taint 키와 같아야 한다)
- **operator** — 값을 어떻게 비교할지
 - **Equal** (기본값): 키와 값이 모두 정확히 같아야 통과. value 필드를 반드시 작성해야 함
 - **Exists**: 키만 같으면 값은 뭐든 통과. value 필드를 쓰면 안됨 (쓰면 에러)
- **value** — 비교할 값
- **effect** — 어떤 효과의 테인트를 통과할지 선택. 비워두면 해당 key의 모든 effect 통과
- **tolerationSeconds** — 출입증의 허용 시간 (NoExecute 테인트 effect에서만 의미)
 - tolerationSeconds가 없을 때는 만료시간 없는 출입증
 - tolerationSeconds가 0 일 때는 즉시 만료되는 출입증

```

tolerations:
- key: "node.kubernetes.io/unreachable"
  operator: "Exists"
  effect: "NoExecute"
  tolerationSeconds: 300 # 5분(300초) 버티다 그 후에 쫓겨
  남

```

실무에서 조심할 몇가지

- 톨러레이션 특수한 조합

- effect를 비우면 해당 키의 모든 효과를 통과(참아냄) → 해당 키의 권한 통행증

```
tolerations:  
- key: "gpu"  
  operator: "Exists"
```

- key까지 비우면 모든 테인트를 통과(참아냄) → 만능 통행증

```
tolerations:  
- operator: "Exists"
```

- 매칭 판정 정확하게 이해하기

스케줄러가 판정하는 방식은 노드의 테인트 중에서 파드의 톨러레이션이 커버하지 못하는 것이 하나라도 존재하면 그 테인트의 효과가 발동한다.

즉, 모든 테인트가 커버되어야 통과된다.

케이스별로 살펴보기 — 노드 테인트 `gpu=true:NoSchedule`

파드의 tolerations	결과	이유
없음	X	출입증이 없음
<code>key=gpu, Equal, value=true, NoSchedule</code>	O	전부 일치
<code>key=gpu, Equal, value=false, NoSchedule</code>	X	Equal인데 value 불일치
<code>key=gpu, Exists, NoSchedule</code>	O	키 일치, 값은 무시
<code>key=gpu, Exists</code> (effect 없음)	O	모든 effect 커버
<code>key=no-gpu, Equal, value=true, NoSchedule</code>	X	key 불일치
<code>key=gpu, Equal, value=true, NoExecute</code>	X	effect 불일치
<code>Exists</code> (key도 없음)	O	만능 통행증

- 테인트가 여러 개일 때는 출입증도 그 수 만큼 있어야 통과

```
kubectl taint nodes k8s-w-1 gpu=true:NoSchedule
```

```
kubectl taint nodes k8s-w-1 team=ai:NoExecute
```

```
tolerations:
- key: "gpu"
  operator: "Equal"
  value: "true"
  effect: "NoSchedule"
- key: "team"
  operator: "Equal"
  value: "ai"
  effect: "NoExecute"
```

- 톨러레이션은 “출입증”이지 “목적지”가 아니다

톨러레이션이 있다고 해서 반드시 그 노드에 가는 것이 아니다. 갈 수 “있게” 될 뿐이다. 그래서 `nodeSelector` 와 세트로 사용한다.

```
kubectl label nodes k8s-w-1 gpu=true # 라벨 (찾아갈 이정표)
kubectl taint nodes k8s-w-1 gpu=true:NoSchedule # 테인트 (출입 통제)
```

```
# 개발자: 파드에 톨러레이션 + 노드셀렉터를 함께 쓴다
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ai-training
spec:
  replicas: 2
  selectors:
    matchLabels:
      app: ai-training
  template:
    metadata:
      labels:
        app: ai-training
    spec:
      containers:
```

```

- name: trainer
  image: tensorflow:latest

tolerations:
- key: "gpu"
  operator: "Equal"
  value: "true"
  effect: "NoSchedule"

nodeSelector:
  gpu: "true"

```

쿠버네티스가 자동으로거는 테인트

우리가 직접 걸지 않아도 쿠버네티스가 자동으로 걸리는 테인트가 존재한다.

테인트 키	언제	effect
<code>node.kubernetes.io/not-ready</code>	노드가 아직 준비 안됨	NoExecute
<code>node.kubernetes.io/unreachable</code>	노드와 연결이 닿지 않음	NoExecute
<code>node.kubernetes.io/memory-pressure</code>	메모리 부족	NoSchedule
<code>node.kubernetes.io/disk-pressure</code>	디스크 부족	NoSchedule
<code>node.kubernetes.io/pid-pressure</code>	프로세스 수 한계	NoSchedule
<code>node.kubernetes.io/network-unavailable</code>	네트워크 미설정	NoSchedule
<code>node.kubernetes.io/unschedulable</code>	kubectl cordon 실행됨	NoSchedule

3. 파드 리소스 관리

파드의 리소스 관리는 결국 **한정된 클러스터 자원을 각 애플리케이션(컨테이너)에 얼마나 적절하게 나누어 사용할 것인가**에 대한 문제이다. 쿠버네티스에서는 각 컨테이너가 필요한 자원의 양을 `requests` 와 `limits` 로 정의할 수 있다.

```
resources:
  requests:
    cpu: "500m"
    memory: "512Mi"
  limits:
    cpu: "1"
    memory: "1Gi"
```

하지만 `requests`, `limits` 를 이해하기 전에 먼저 쿠버네티스가 어떤 리소스를 관리할 수 있으며 각각 어떤 단위를 사용하는지 알아볼 필요가 있다.

1) 쿠버네티스의 리소스 관리 목록

Pod는 리소스 요청과 한도를 통해서 리소스를 관리할 수 있다. 책에서 파드의 리소스 관리는 `cpu`, `memory`만 가능하다고 나와 있지만 현재 기준에서는 `cpu`, `memory`, `ephemeral-storage` (disk for pod, v1.25+ GA), `hugepages-1Gi` (v1.14+ GA), `hugepages-2Mi` (v1.14+ GA)가 가능하다.

CPU 단위 — 코어 개수

```
cpu: "1" # 코어 1개 (= 1vCPU)
cpu: "2" # 코어 2개
cpu: "0.5" # 코어 0.5개
cpu: "500m" # 코어 0.5개
cpu: "100m" # 코어 10분의 1
```

`m`은 밀리(milli), 1000분의 1이다. 따라서 $1000m = 1$ 이다.

“코어 0.5개”는 무슨 의미일까? `500m`는 CPU 코어를 물리적으로 반으로 잘라서 주는 것이 아니다. CPU는 운영체제가 여러 프로세스에게 실행 시간을 나눠준다. 따라서 `500m`는 CPU 1개가 제공하는 처리 능력의 약 절반에 해당하는 CPU 시간을 의미한다.

Memory 단위 — 바이트

접미사	배수	예
<code>Ki</code> , <code>Mi</code> , <code>Gi</code> , <code>Ti</code>	1024배씩	1Mi = 1,048,576 바이트
<code>k</code> , <code>M</code> , <code>G</code> , <code>T</code>	1000배씩	1M = 1,000,000 바이트

쿠버네티스에서는 `Mi`, `Gi`를 쓰는 것이 관례이다.

주의할 것은 메모리에 소문자 **m** 을 쓰게되면 밀리바이트로 해석된다. 즉 0.5 바이트를 요청하게 된다.

문법적으로 유효하기 때문에 에러가 나지는 않지만 limits에 설정했을 경우 OOM이 발생할 가능성이 높다

Ephemeral Storage(Pod를 위한 노드의 임시 디스크 공간) 단위 — 바이트

Ephemeral Storage도 바이트 단위이기 때문에 Mi, Gi 와 함께 M, G 등도 사용가능하다. 주로 Mi, Gi 등을 쓰는 것이 관례이다.

```
ephemeral-storage: "500Mi"
ephemeral-storage: "1Gi"
ephemeral-storage: "2Gi"
ephemeral-storage: "10Gi"
```

HugePages-* 단위 — 바이트

hugepages-2Mi, hugepages-1Gi는 디스크가 아니라 RAM이다. 운영체제는 RAM을 관리할 때 메모리를 작은 Page 단위로 나눠서 관리한다. 일반적인 Linux 시스템에서는 Page 당 4KiB 정도로 메모리를 관리한다.

예를 들어 1GiB 메모리를 4KiB 단위로 관리하면 262,144개의 페이지로 관리해야한다. 고성능 애플리케이션에서는 이 페이지 관리 자체가 부담이 될 수 있기 때문에 훨씬 큰 페이지를 사용할 수 있게 만든 것이 HugePage이다.

```
hugepages-2Mi: "100Mi"
```

- **2Mi** → 페이지 하나의 크기
- **100Mi** → 총 필요한 HugePage 메모리 양

→ $100\text{Mi} / 2\text{Mi} = 50\text{개}$

노드의 실제 가용량 — Capacity(물리적 총량)와 Allocatable(파드가 쓸 수 있는 양)

Pod가 리소스를 요청한다고 해서 Node가 가지고 있는 자원을 전부 Pod에게 줄 수 있는 것은 아니다. Node에서는 운영체제, kubelet, container runtime 및 기타 시스템 프로세스도 CPU와 Memory 등을 사용해야 한다. 그래서 Kubernetes에서는 Node 리소스를 크게 두 가지 관점으로 보여준다.

- **Capacity** = Node가 가지고 있는 전체 자원

- **Allocatable** = 그 중 Pod에게 할당 할 수 있는 자원

```
Capacity:
  cpu:                4
  ephemeral-storage:  50620216Ki
  hugepages-1Gi:      0
  hugepages-2Mi:      0
  memory:              16373052Ki
  pods:               110
Allocatable:
  cpu:                4
  ephemeral-storage:  49243346087
  hugepages-1Gi:      0
  hugepages-2Mi:      0
  memory:              16373052Ki
  pods:               110
```

```
kubectl describe node k8s-w-1
kubectl describe node <노드이름>
```

2] requests & limits

위에서 살펴본 CPU, Memory, Ephemeral Storage, HugePages를 각 컨테이너에 얼마나 배정할 것인지 정의하는 방법이 **requests** 와 **limits** 이다. 둘의 역할을 한 문장으로 구분하면 다음과 같다.

- requests = 이 컨테이너를 실행하기 위해 이 정도 자원을 예약해 주세요.
- limits = 실행된 뒤 이 자원은 여기까지만 사용할 수 있습니다.

```
kubectl top nodes
```

NAME	CPU(cores)	CPU(%)	MEMORY(bytes)	MEMORY(%)
k8s-cp-1	43m	2%	1630Mi	20%
k8s-w-1	8m	0%	927Mi	5%

	requests: 요청, 예약	limits: 상한, 한도
누가 보나	스케줄러 (배치할 때)	커널/cgroup (실행 중에)
언제 쓰이나	파드를 어디 노드에 넣을지 정할 때 딱 한 번	파드가 도는 내내 계속 감시
초과하면	(초과라는 개념이 없음)	제재
안 쓰면	스케줄러가 0으로 봄	노드 자원을 무한정 쓸 수 있음

requests — 스케줄링을 위한 예약량

Scheduler는 하나의 Pod를 Node에 배치할 때 Pod 내부 Container들의 `requests` 를 합산하여 해당 Node가 Pod 전체를 수용할 수 있는지 판단한다.

- Node의 Allocatable Memory = 8Gi 이고
- Node에 현재 배치된 Pod들의 memory request 합계가 6Gi 라면
- request 기준 남은 공간은 $8Gi - 6Gi = 2Gi$ 가 된다.

만약 새 Pod가 3Gi를 요구한다면 request를 기준으로 해당 Node에 스케줄링이 불가능하다.

반대로 1Gi를 요구한다면 request를 기준으로 해당 Node에 배치할 수 있다.

실무에서 유의할 점 — 스케줄러는 “실제 사용량”을 보지 않고 requests의 합계만 본다

각 Pod의 requests를 실제 사용량보다 너무 여유롭게 잡는다면, 실제 자원은 놓고 있지만 새로운 Pod가 추가될 때 Pod가 안 올라오는 상황이 생길 수 있다는 것을 기억하자.

limits — 실행 중 사용할 수 있는 상한

Pod가 Node에 배치된 뒤에는 `limits` 가 중요해진다.

다만 리소스 종류에 따라 limit을 초과했을 때 동작이 다르다.

Resource	Limits 초과 시
CPU	Throttling — 프로세스를 죽이지 않고 CPU 사용 속도를 제한
Memory	OOM Kill 가능 — 컨테이너 프로세스가 종료될 수 있음
Ephemeral Storage	Pod Eviction 가능
HugePages	미리 확보한 HugePage 이상 사용 불가

3] requests & limits 실습해보기

▼ CPU request

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: cpu-request
spec:
  replicas: 1
```

```
selector:
  matchLabels:
    app: cpu-request

template:
  metadata:
    labels:
      app: cpu-request

  spec:
    containers:
      - name: nginx
        image: nginx:alpine

        resources:
          requests:
            cpu: "12"
```

▼ Memory request

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: memory-request
spec:
  replicas: 1

  selector:
    matchLabels:
      app: memory-request

  template:
    metadata:
      labels:
        app: memory-request

    spec:
      containers:
```

```
- name: nginx
  image: nginx:alpine

  resources:
    requests:
      memory: "16Gi"
```

▼ CPU limit - Throttling 실습용 Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: cpu-test
spec:
  replicas: 1

  selector:
    matchLabels:
      app: cpu-test

  template:
    metadata:
      labels:
        app: cpu-test

    spec:
      containers:
        - name: cpu-eater
          image: alpine:3.20

          resources:
            requests:
              cpu: "50m"
              memory: "16Mi"

            limits:
              cpu: "100m"
              memory: "32Mi"
```

```
command:
  - sh
  - -c
  - |
    while true; do
      :
    done
```

```
kubectl top pod
```

무한 루프임에도 불구하고 100m 정도에서 머무는 것을 확인할 수 있다.

▼ Memory limit - OOM 실습용 Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: memory-test
spec:
  replicas: 1

  selector:
    matchLabels:
      app: memory-test

  template:
    metadata:
      labels:
        app: memory-test

    spec:
      containers:
        - name: memory-eater
          image: python:3.12-alpine

          resources:
```

```

requests:
  cpu: "100m"
  memory: "64Mi"

limits:
  cpu: "500m"
  memory: "128Mi"

command:
- python
- -c
- |
  import time

  data = []

  while True:
    data.append(bytearray(10 * 1024 * 102
4))

    print(f"Allocated approximately {len(d
ata) * 10} MiB", flush=True)
    time.sleep(1)

```

```

kubectl get pods -w
kubectl logs <pod> --previous
kubectl describe pod <pod> # Containers > Last State의 00
MKilled 확인

```

4. QoS: Quality of Service

Pod를 만들게 되면 쿠버네티스가 자동으로 Pod의 QoS 등급을 매기게 된다. 이는 Node의 CPU나 Memory가 부족해지는 상황에서 "어떤 Pod를 먼저 희생시키고, 어떤 Pod를 최대한 보호할 것인가?" 라는 판단이 필요하기 때문이다.

물론 QoS만으로 판단하는 것이 아니라 Pod Priority, 실제 사용량, request를 얼마나 초과해서 쓰고 있는지 등을 함께 고려해서 실제 eviction을 계산하게 된다.

제일 오래 살아남는 순서

1. Guaranteed, 그리고 requests 이내로 쓰고 있는 Burstable
2. Burstable 중 requests를 초과해 쓰는 것 (초과량 큰 순)
3. BestEffort

1] Guaranteed — 필요한 만큼 정확하게 예약

Pod 안의 모든 Container에 **CPU**와 **Memory**의 requests와 limits이 설정되어 있어야 하며 **requests의 값과 limits의 값이 동일**해야 한다.

다만 limits만 쓰고 requests를 생략하면, 쿠버네티스가 requests를 limits와 같은 값으로 자동으로 채워준다. 그래서 이것도 Guaranteed가 된다.

▼ Deployment YAML

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-qos-guaranteed

spec:
  replicas: 1

  selector:
    matchLabels:
      app: nginx-qos-guaranteed

  template:
    metadata:
      labels:
        app: nginx-qos-guaranteed

    spec:
      containers:
        - name: nginx
          image: nginx:alpine
          ports:
            - containerPort: 80

          resources:
```

```

        requests:
          cpu: "200m"
          memory: "128Mi"
        limits:
          cpu: "200m"
          memory: "128Mi"

- name: nginx-monitor
  image: busybox:1.36
  command:
    - sh
    - -c
    - |
        while true; do
          echo "$(date) checking nginx..."
          wget -q -O /dev/null http://127.
0.0.1 && \
              echo "nginx is alive" || \
              echo "nginx is down"
          sleep 5
        done

  resources:
    requests:
      cpu: "50m"
      memory: "32Mi"
    limits:
      cpu: "50m"
      memory: "32Mi"

```

requests & limits

```

- name: nginx
  image: nginx:alpine
  ports:
    - containerPort: 80

```

```

    resources:
      requests:
        cpu: "200m"
        memory: "128Mi"
      limits:
        cpu: "200m"
        memory: "128Mi"

- name: nginx-monitor
  image: busybox:1.36

  command:
  - sh
  - -c
  - |
    while true; do
      wget -q -O /dev/null http://127.0.0.1
      sleep 5
    done

  resources:
    requests:
      cpu: "50m"
      memory: "32Mi"
    limits:
      cpu: "50m"
      memory: "32Mi"

```

limits

```

- name: nginx
  image: nginx:alpine
  ports:
  - containerPort: 80

  resources:
    limits:
      cpu: "200m"

```

```

        memory: "128Mi"

-   name: nginx-monitor
    image: busybox:1.36

    command:
    - sh
    - -c
    - |
        while true; do
            wget -q -O /dev/null http://127.0.0.1
            sleep 5
        done

    resources:
        limits:
            cpu: "50m"
            memory: "32Mi"

```

2] Burstable — 최소 이 정도는 필요하지만 여유가 있으면 더 사용

requests나 limits를 지정했지만 Guaranteed의 조건을 못 채운 경우, 즉 일치하지 않는 경우

```

# requests < limits (전형적인 Burstable)
resources:
  requests:
    memory: "150Mi"
  limits:
    memory: "200Mi"

# requests만 있고 limits 없음
resources:
  requests:
    cpu: "500m"

# CPU, Memory 중 하나만 requests == limits 지정
resources:
  requests:

```

```

    cpu: "500m"
  limits:
    cpu: "500m"

```

- 컨테이너 하나라도 request나 limit이 있다면 웬만하면 Burstable

3] BestEffort — 아무 예약도 안함

```

containers:
- name: nginx
  image: nginx:alpine

- name: nginx-monitor
  image: busybox:1.36
  command:
  - sh
  - -c
  - |
    while true; do
      wget -q -O /dev/null http://127.0.0.1
      sleep 5
    done

```

확인 방법

```

kubectl get pods -l app=nginx-qos-guaranteed -o jsonpath=
='{.items[0].status.qosClass}'
kubectl get pods -l <label> -o jsonpath='{.items[0].status.
qosClass}'
# Guaranteed%

kubectl describe pods -l app=nginx-qos-guaranteed | grep "Q
oS Class"
kubectl describe pods -l <label> | grep "QoS Class"
# QoS Class: Guaranteed

```

실무 가이드 라인 요약

메모리: requests와 limits 를 함께

메모리는 압축 불가능한 자원이라 초과하면 죽는다. 넉넉하게 예약해두고, 그 안에서 쓰도록 제한하자.

→ 내가 예약한 만큼만 쓰니 남 때문에 죽을 일도, 나 때문에 남이 죽을 일이 없다.

CPU: requests는 반드시, limits는 신중히

CPU는 넘어도 죽지는 않고 다만 느려질 뿐이다. 서비스의 성격에 따라 고민해보자. 다만 limits를 생략하면 QoS가 Burstable로 되는 것을 기억하자

Pod도 제한이 존재한다

```
kubectl describe node k8s-w-1
kubectl describe node <node-name>
```

```
Allocatable:
  cpu: 4
  ephemeral-storage: 49243346087
  hugepages-1Gi: 0
  hugepages-2Mi: 0
  memory: 16373052Ki
  pods: 110
```

- `pods: 110` 도 스케줄링 대상 자원이다. CPU, Memory가 여유 있어도 Pods 개수가 찼다면 Too many pods로 Pending이 난다.

5. PDB: PodDisruptionBudget

Pod의 중단(Disruption)을 어디까지 허용할 것인가에 대한 예산(Budget)이다. 주로 drain하고 함께 사용된다.

예를 들어 Node의 유지보수를 위해 Node를 하나씩 내려야 하는 상황이 생길 수 있다. 그럴 때는 drain을 사용한다.

```
kubectl drain k8s-w-1
```

Drain은 Node에 있는 Pod를 Eviction 하려고 한다.

그런데 아무 제한에 없다면 분산되어 있는 여러 nginx Pod가 동시에 내려가 nginx의 가용성이 떨어질 수 있다.

이때 PDB를 설정하여 최소 Pod 수, 최대 Pod 수 등을 지정할 수 있다.

1] minAvailable → 살아있어야 하는 수

최소한 몇 개는 반드시 사용 가능한 상태여야 하는가?

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: nginx-pdb-min-available

spec:
  minAvailable: 2

  selector:
    matchLabels:
      app: nginx
```

```
kubectl get pdb
```

예를 들어 Pod가 3개가 있다면 2개는 살아야 한다 = Pod 2개는 살아있어야 함

2] maxUnavailable → 죽어 있어도 되는 수

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: nginx-pdb-max-unavailable

spec:
  maxUnavailable: 2

  selector:
    matchLabels:
      app: nginx
```

```
kubectl get pdb
```

예를 들어 Pod가 3개 있다면 1개만 죽을 수 있다 = Pod 2개는 살아있어야 함

사실상 `spec.replicas: 3` 이면 같은 의미이다. 하지만 본래 표현하고자 하는 정책이 다른 것이다.

- 나중에 replicas: 100이 되었다고 한다면,
- minAvailable: 2 = 98개까지 축출 가능
 - maxUnavailable: 1 = 1개만 축출 가능

3] Percentage도 가능하다

- minAvailable: 80%
- maxUnavailable: 25%

PDB에 대해 주의할 점

PDB는 자발적 축출(Voluntary eviction)에서만 주로 활용된다.
비자발적 축출과 같은 경우에는 PDB는 막을 수 없다.

6. Namespace와 리소스 정책

1] Namespace: 논리 구역

쿠버네티스 클러스터 안에는 여러 애플리케이션과 팀의 리소스가 존재할 수 있다. 모든 Deployment, Service, ConfigMap 등을 하나의 공간에 넣어 관리하면 이름 충돌이나 권한 관리, 리소스 사용량 관리가 어려워질 수 있다.

쿠버네티스는 이를 위해 Namespace라는 논리적인 구역을 제공한다. 또한 쿠버네티스가 기본적으로 제공하는 Namespace가 존재한다.

```
kubectl get namespaces
kubectl get ns
```

NAME	STATUS	AGE
default	Active	243d
k8s-lab	Active	4d3h
kube-node-lease	Active	243d
kube-public	Active	243d
kube-system	Active	243d

- **kube-node-lease** : Node의 heart-beat 기록용
- **kube-public** : 인증 없이도 읽을 수 있는 정보를 두는 곳
- **kube-system** : 클러스터의 부품 즉 coredns, traefik, metrics-server 등이 여기에 존재한다.
- **default** : Namespace를 지정하지 않았을 때 기본적으로 들어가는 NS.
ResourceQuota, LimitRange, RBAC, NetworkPolicy, PodSecurity 등 아무것도 존재하지 않는다.

Namespace 생성하기

- 선언형 YAML

```
apiVersion: v1
kind: Namespace
metadata:
  name: payment-team
```

- 명령형

```
kubectl create namespace payment-team
```

Deployment에 Namespace 적용하기

```
apiVersion: apps/v1
kind: deployment
metadata:
  name: nginx
  namespace: payment-team

spec:
  replicas: 2

...
```

```
kubectl get deployment -n payment-team
kubectl get pods -n payment-team
```

"default" Namespace

Namespace를 지정하지 않으면 기본적으로 `default` Namespace를 사용한다.

```
# 기본 namespace를 payment-team으로 변경
kubectl config set-context --current --namespace=payment-team

# 현재 기본 namespace 확인
kubectl config view --minify | grep namespace
```

실무적으로 기억해야할 사항

- Namespace가 다르면 같은 이름을 사용할 수 있다.
- Namespace는 Node를 물리적으로 분리하는 기능이 아니다. **논리적인 분리**일 뿐이다.
- 실무 설계 — Namespace를 어떻게 나눌까?
 - **환경별(dev / staging / prod)로 나누는 것은 유의하자** → 같은 클러스터라면 같은 노드를 공유
 - dev 파드에 버그가 있어 메모리 폭주가 일어난다면 prod 파드에도 영향을 미친다.
 - 운영(prod)는 별도의 클러스터로 분리하는 것이 원칙이다.
 - 부득이하게 한 클러스터에 뒀야 한다면 Taint + NodeSelector로 노드를 물리적으로 갈라놓자
 - **팀별 / 서비스별로 나누는 것은 좋은 선택이다.**
 - 여러 팀이 한 클러스터를 쓴다 → 팀별(payments, search, platform) RBAC 경계가 팀 경계와 일치
 - 한 팀이 클러스터를 독점한다 → 서비스/도메인별 (order, catalog, auth)
 - **Namespace의 네이밍 컨벤션은 처음에 정하자** → 나중에 바꾸기가 거의 불가능
 - <팀>-<서비스>-<환경>
 - <서비스>-<환경>

Namespace를 실질적으로 사용하는 이유: 다양한 정책 일괄 적용

- ResourceQuota → 할당량
- LimitRange → 자원제한
- RBAC → 권한
- NetworkPolicy → 통신제어
- PodSecurity → 보안

여기서는 ResourceQuota와 LimitRange를 살펴볼 예정이다.

2] ResourceQuota

이제 팀 별로 Namespace를 나누었다면 무작정 리소스를 사용하게 만들지는 않을 것이다. 우리 팀이 클러스터의 모든 리소스를 다 사용한다면, 다른 팀이 올릴 Pod가 존재하지 않는

다.

따라서 각 NS별로 전체 사용량을 제한할 수 있으며 이를 **ResourceQuota** 라고 한다.

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: payment-team-quota
  namespace: payment-team

spec:
  hard:
    requests.cpu: "2"
    requests.memory: "4Gi"

    limits.cpu: "4"
    limits.memory: "8Gi"

  pods: "10"
```

6-2-payment-team-quota.yml

- **requests.cpu** — payment-team NS의 Pod들의 requests.cpu 합계
- **requests.memory** — payment-team NS의 Pod들의 requests.memory 합계
- **limits.cpu** — payment-team NS의 Pod들의 limits.cpu 합계
- **limits.memory** — payment-team NS의 Pod들의 limits.memory 합계

```
kubectl apply -f 6-2-payment-team-quota-all.yml
kubectl get quota -n payment-team
```

NAME	REQUEST	LIMIT	AGE
payment-team-quota	pods: 0/10, requests.cpu: 0/2, requests.memory: 0/4Gi	limits.cpu: 0/4, limits.memory: 0/8Gi	19s

```
kubectl apply -f 6-2-payment-team-nginx.yml
```

3] LimitRange

Namespace에 ResourceQuota로 requests, limits를 적용하게 되면 Pod의 Container들에 반드시 requests, limits를 작성해주어야 한다.

하지만 개발자들이 깜박 넣지 않아 아래와 같은 에러가 발생할 수 있다.

```
Error creating: pods "payment-team-nginx-647dbd7bcd-mv96q"
is forbidden:
failed quota: payment-team-quota: must specify limits.cpu f
or: nginx;
limits.memory for: nginx
```

requests, limits를 설정한 quota에 limits를 작성하지 않은 Pod의 경우

또한 Namespace를 나눴다고 해도 팀원 모두 마음대로 리소스를 요청할 수 있다면 문제가 생길 수 있다.

```
resources:
  requests:
    memory: "100Gi"
```

LimitRange는 하나의 Namespace 안에서 개별 Container, Pod 등의 리소스 사용 규칙을 정의하는 리소스이다.

LimitRange를 통해서 아래와 같이 설정할 수 있다.

- 기본 requests, limits
- 최소, 최대 리소스
- requests와 limits 비율

기본 requests 와 limits 설정

```
apiVersion: v1
kind: LimitRange
metadata:
  name: payment-team-limit-range
  namespace: payment-team

spec:
  limits:
```

```

- type: Container

  defaultRequest:
    cpu: "100m"
    memory: "128Mi"

  default:
    cpu: "500m"
    memory: "512Mi"

```

- `defaultRequest` → resources.requests의 기본값
- `default` → resources.limits의 기본값

```

kubectl apply -f 6-3-payment-team-limit-range.yml
kubectl get limits -n payment-team

```

```

kubectl apply -f 6-3-nginx.yml

```

```

kubectl describe pod <pod-name>
kubectl get quota -n payment-team

```

NAME	REQUEST	LIMIT
payment-team-quota	pods: 1/10, requests.cpu: 1/2, requests.memory: 1Gi/4Gi	limits.cpu: 2/4, limits.memory: 2Gi/8Gi

최소 & 최대

```

apiVersion: v1
kind: LimitRange
metadata:
  name: payment-team-limit-range-min-max
  namespace: payment-team

spec:
  limits:
    - type: Container

    min:

```

```
cpu: "50m"
memory: "64Mi"

max:
  cpu: "2"
  memory: "2Gi"
```

```
kubectl apply -f 6-3-payment-team-limit-range-min-max.yml
kubectl apply -f 6-3-nginx-max.yml
```

일반적으로 기본값 설정을 같이 넣는 것이 좋지만, 만약 `min`, `max` 범위만 LimitRange에 넣었을 경우 `max`의 값이 **defaultRequest, default 값처럼 상정된다.**

min만 있을 경우에는 defaultRequest만 커버하여 Quota에 limits 설정되어 있는 경우에는 Pod가 올라가지 않는다.

- `min` 만 있을 경우 → defaultRequest 커버
- `max` 만 있을 경우 → defaultRequest, default 커버
- `default` 만 있을 경우 → defaultRequest, default 커버

그래도 예상치 못한 상황이 발생할 수 있으니 웬만하면 다 적어주자...

7. 쿠버네티스 스케일링

쿠버네티스에서 "스케일링을 한다"는 것은 크게 세 가지로 나눌 수 있다.

- **Node Autoscaling** → Node 개수 조절
- **Horizontal Pod Autoscaler, HPA** → Pod 개수 조절
- **Vertical Pod Autoscaler, VPA** → Pod의 리소스 조절

1) 노드 스케일링: Node Scaling

책에서는 클러스터 스케일링, 클러스터 오토스케일링이라고 표현하지만 최근에는 노드 스케일링으로 표현하는 것이 더 정확한 것 같다.

Node Autoscaling

Automatically provision and consolidate the Nodes in your cluster to adapt to demand and optimize cost.

 <https://kubernetes.io/docs/concepts/cluster-administration/node-autoscaling/>

kubern

노드 스케일링은 Pod 수가 아니라 Node 수를 늘리거나 줄이는 것을 의미한다.

즉 Node를 몇 대 운영할 것인가를 고민하는 것이다.

예를 들어 클러스터 전체가 제공할 수 있는 CPU/Memory/Pod 수 같은 클러스터의 가용 컴퓨팅 자원이 부족하다면 Node를 추가할 수 있다. 이것이 Scale Out 이다.

반대로 애플리케이션의 부하가 줄어들어 클러스터의 자원 수요가 감소하면, 불필요한 Node를 제거할 수 있다. 이것이 Scale In이다.

수동 노드 스케일링

사람이 직접 Node를 추가하는 방법

kubeadm

새 서버에 container runtime, kubelet, kubeadm 등을 설치

이후에 아래와 같이 kubeadm join 사용

```
sudo kubeadm join 192.168.0.10:6443 \
  --token abcdef.1234567890abcdef \
  --discovery-token-ca-cert-hash sha256:xxxx...
```

k3s

```
curl -sL https://get.k3s.io | \
  K3S_URL=https://<cp-ip>:6443 \
  K3S_TOKEN='<cp-node-token>' \
  sh -
```

자동 노드 스케일링: Node Autocaling

수동으로 매번 Node를 추가하는 것은 앞선 판단도 필요하기 때문에 신경 쓸 요소들이 많아진다. 쿠버네티스는 자동으로 Node 추가할 수 있는 기능들이 존재한다. 대표적인 구현체로는 Cluster Autoscaler와 Karpenter가 존재한다.

- **Cluster Autoscaler**

미리 만들어놓은 Node Group 중 Pending Pod를 확인하고 적절한 Node Group의 Node 생성

- **Karpenter**

이 Pending Pod에 필요한 Node가 뭔지 확인하고 적당한 VM 자체 생성

어떤 방식이든 중요한 것은 새로운 Pod가 생성되었을 때 적당하게 들어갈만한 Node가 존재하지않아 **Pending / Unschedulable** 가 되었을 때 **Node**를 추가한다는 것이다.

중요한 것은 실제 CPU 사용률이나 Memory 사용률이 아니라 requests 등을 통해 **스케줄러가 넣을 수 있는지 없는지로 판단하는 것이다.**

반대로 Node가 너무 많아진 상황을 아래와 같이 상정해보자면

```
Node A Memory - 80%
Node B Memory - 10%
Node C Memory - 5%
Node D Memory - 3%
```

Node C와 D가 없어도 모든 Pod를 Node A/B에 재배포할 수 있다면 Node Autoscaler가 Node 줄이는 것을 검토할 수 있다.

- Node C/D → Pod eviction
- Node A/B → Pod 재생성
- Node C/D 제거

여기서 앞서 살펴보았던 PDB가 연결된다. Node를 없애려면 먼저 Node의 Pod를 다 내보내야하며 **PDB(PodDisruptionBudget)**를 통해 **최소 가용 Pod가 깨지는지 확인하며 너무 많은 Pod가 내려가지 않도록 보호한다**

2] 애플리케이션 스케일링

이제는 Node가 아니라 Pod와 관련해서 세팅을 하는 것이다.

기본적으로 Deployment 리소스를 사용하여 Pod의 개수를 지정할 수 있다.

또한 수동으로 kubectl 명령 등을 통해 Pod의 개수를 키우거나 줄일 수 있다.

```
kubectl apply -f 7-2-nginx-deployment.yml
```

nginx replicas: 3

```
kubectl scale deployment deployment-test --replicas=5  
kubectl get pods
```

수동으로 하는 것도 좋지만 수동으로 하게되면 놓치거나 실수하게 되는 휴먼 에러가 발생하기 쉽다. 따라서 쿠버네티스는 메트릭 등을 보면서 자동으로 확장하는 Pod 관련 오토스케일러를 제공한다.

HPA(Horizontal Pod Autoscaler)

수평 파드 오토스케일링

쿠버네티스에서, HorizontalPodAutoscaler 는 워크로드 리소스(예: 디플로이먼트 또는 스테이트풀셋)를 자동으로 업데이트하며, 워크로드의 크기를 수요에 맞게 자동으로 스케일링하는 것을 목표로 한다. 수

 https://kubernetes.io/ko/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/?utm_source=chatgpt.com

kubern

공식문서를 살펴보면 HPA는 Deployment, StatefulSet 같은 리소스의 Pod 크기를 조절할 수 있다. 반면 크기 조절이 불가능한 오브젝트(DemonSet)에는 적용할 수 없다.

예를 들어서 사용자가 많아지면서 Pod의 CPU 사용량이 올라가면 등록된 HPA가 Deployment의 replica 수를 증가시킬 수 있다

▼ deployment.yml

```
apiVersion: apps/v1  
kind: Deployment  
metadata:  
  name: nginx  
  
spec:  
  replicas: 3  
  
  selector:  
    matchLabels:  
      app: nginx
```

```

template:
  metadata:
    labels:
      app: nginx

  spec:
    containers:
      - name: nginx
        image: nginx:alpine

    resources:
      requests:
        cpu: "500m"
        memory: "128Mi"

      limits:
        cpu: "1"
        memory: "256Mi"

```

책과는 다르게 현재는 `autoscaling/v2` 를 사용한다.

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: nginx-hpa

spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: nginx

  minReplicas: 3
  maxReplicas: 10

  metrics:
    - type: Resource

```

```
resource:
  name: cpu
  target:
    type: Utilization
    averageUtilization: 50
```

- `scaleTargetRef` — 누구를 스케일링 할 것인가
- `minReplicas` — 최소 개수
- `maxReplicas` — 최대 개수
- `averageUtilization: 50`
 - 각 Pod 마다 계산

$$\frac{\text{실제 CPU Usage}}{\text{CPU Request}} \times 100$$

request를 잘못 잡으면 HPA가 의도하지 않을 방향으로 실행될 수 있다.

- HPA가 해당 Pod들의 평균을 가지고 필요한 replicas를 계산

$$desiredReplicas = \text{ceil} \left[currentReplicas \times \frac{currentMetricValue}{desiredMetricValue} \right]$$

만약 Container가 여러 개 있는 Pod들은 어떻게 해야할까?

- `type: Resource`
CPU metric은 Pod 내부 컨테이너 resource 사용량이 합쳐진다.
따라서 특정 컨테이너가 높아도 평균의 법칙으로 낮아보일 수도 있다.
그래서 특정 컨테이너만 바라보는 타입이 존재한다.
- `type: ContainerResource` : 특정 컨테이너만 확인 (1.30+GA)

```
metrics:
- type: ContainerResource
  containerResource:
    name: cpu
    container: nginx
    target:
      type: Utilization
      averageUtilization: 60
```

containers.name과 맞춰야한다.

만약 `requests.cpu` 가 없으면 어떻게 될까?

"Please note that if some of the Pod's containers do not have the relevant resource request set, CPU utilization for the Pod will not be defined and the autoscaler will not take any action for that metric."

- `type: Utilization` 의 경우 HPA가 반응하지 않는다.
- `type: AverageValue` 의 경우 usage의 평균만 확인하기 때문에 HPA 가능

1. 엄밀하게 말하면 HPA가 직접 Pod를 만드는 것은 아니다.

HPA는 부하를 통해 scale 대상의 desired replica 수를 변경하는 것이다.

2. HPA는 여러 종류의 metric을 기준으로 Pod 수를 조절할 수 있다.

- CPU
- Memory
- Custom Metrics
- External Metrics

3. HPA는 CPU/Memory 사용량을 직접 측정하지 않는다.

Kubernetes 공식 Resource Metrics Pipeline

- kubelet이 컨테이너의 resource metric을 제공
- Metrics Server가 각 kubelet에서 메트릭을 수집·집계하여 Metrics API로 노출
- HPA가 Metrics API로 노출된 값을 사용

- `--horizontal-pod-autoscaler-sync-period=15s`

HPA가 몇 초마다 한 번씩 계산을 수행할 것인가?

- Default: 15s

- `--horizontal-pod-autoscaler-tolerance=0.1`

목표값에서 조금 벗어난 것까지 일일이 스케일링하지 않도록 하는 오차 허용 범위

- Default: 0.1

- `--horizontal-pod-autoscaler-downscale-stabilization=5m`

해당 기간 동안 과거 recommendation을 살피고 그보다 낮게 scale down 하지 않도록 한다. (급격하게 줄어드는 것 방지)

- Default: 5m
- 과거의 horizontal-pod-autoscaler-downscale-delay 역할

- `--horizontal-pod-autoscaler-cpu-initialization-period=5m`

새 Pod가 시작될 때 발생하는 비정상적으로 높은 사용량을 진짜 부하라고 착각하지 않도록 하기 위한 시간 설정

- Default: 5m

- `--horizontal-pod-autoscaler-initial-readiness-delay=30s`

Pod 시작 직후 초기 readiness 상태변화 용인하여 설불리 Pod를 늘리지 않도록 하기 위한 시간 설정

VPA(Vertical Pod Autoscaler)

현재 VPA는 K8s 기본 core API에 포함되어 실행되는 것이 아니라 **별도로 설치해야 하는 add-on**이다.

Vertical Pod Autoscaling

In Kubernetes, a VerticalPodAutoscaler automatically updates a workload management resource (such as a Deployment or StatefulSet), with the aim of automatically

 <https://kubernetes.io/docs/concepts/workloads/autoscaling/vertical-pod-autoscale/>

kubern

VPA는 Pod의 CPU/Memory request/limit을 조절한다.

- **Recommender:** 이 Container는 어느 정도 자원이 적당할지 추천

현재/과거 사용량 CPU, Memory, OOM 발생 여부 등을 모니터링하여 권장값을 추천

- **Updater:** Pod 관리 결정

현재 할당값과 추천값의 차이, 업데이트 가능 여부, PDB 등 조건을 고려해 실제로 업데이트할 Pod를 결정

- 전통적인 Recreate 방식 → Pod를 Eviction

- **Admission Controller:** 새 Pod 생성 시 추천값 적용